HADES

Web Security Scanner
Every Module Explained — Plain & Simple

For authorised security testing only. Scanning systems without explicit written permission is illegal. For each of the 43 scan modules — and the dedicated Database, AI/LLM, Engagement, Out-of-Band, CVE Intelligence and TLS profiles — this guide explains, in plain language: **what it checks**, the **consequence of a finding**, and the **type of attack** it enables.

Generated 2026-06-06

How to read this guide

Hades runs many small **modules**, each checking one thing about a website. Every module card below has the same three parts: **What it checks** (in plain words), **Consequence of a finding** (what is at risk), and **Possible attack** (how it is abused). No prior knowledge needed — terms are explained as they appear.

Severity levels

Level	Meaning
INFO	Just information / context. Never affects the score.
LOW	Minor hardening gap; little direct risk on its own.
MEDIUM	Worth fixing; helps attackers or leaks information.
HIGH	Serious; a likely path to compromise.
CRITICAL	Severe; often a direct way in (secrets, code execution).

1 · Reconnaissance Modules (11)

basic_info

Target Fingerprint

What it checks: Makes a first request and records the essentials: IP address, web-server software, load time, page title, and a guess of the operating system from the network TTL value.

Consequence of a finding: On its own harmless, but it is the attacker's very first step — knowing the server and OS narrows down which known weaknesses to try.

Possible attack: Reconnaissance / fingerprinting — the groundwork for every targeted exploit that follows.

Good to know: Everything here is INFO and never lowers your score — it is context, not a problem.

whois_lookup

Domain Registration Lookup

What it checks: Queries public WHOIS databases for who registered the domain, its creation/expiry dates and name servers (skips meaningless platform sub-domains).

Consequence of a finding: A domain about to expire can be lost and re-registered by anyone; registration details feed convincing social engineering.

Possible attack: Domain hijacking (on expiry) and targeted phishing using real registration data.

Good to know: An expiry date that is very close is flagged because an expired domain can be taken over.

dns_check

Email Security Records (DNS)

What it checks: Looks up the DNS records that protect email — SPF, DKIM, DMARC — and the MX mail servers.

Consequence of a finding: Missing SPF/DMARC means nothing proves an email truly came from your domain, so anyone can forge it.

Possible attack: Email spoofing, phishing and CEO-fraud sent from your own domain name — one of the most common real-world weaknesses.

Good to know: Missing SPF/DMARC does not break the website, but it lets criminals impersonate your email address.

ssl_check

TLS / Certificate Inspector

What it checks: Examines the HTTPS certificate: issuer, TLS version, days until expiry, and whether the site is served over plain HTTP with no encryption at all.

Consequence of a finding: No HTTPS exposes all traffic (passwords included) to anyone on the network; an expired certificate breaks trust and warns visitors away.

Possible attack: Man-in-the-middle interception and traffic tampering on shared/public networks.

Good to know: A certificate expiring soon is flagged early so it can be renewed before browser warnings appear.

port_scan

TCP Port Scanner

What it checks: Opens connections to ~26 common service ports (databases, RDP/VNC/SSH, web, mail) and grabs a banner to confirm the service; first probes random ports to detect an 'accept-all' host.

Consequence of a finding: An exposed database or remote-desktop port is a direct doorway into the server that should never face the internet.

Possible attack: Brute-force of the exposed service, or exploitation of the service to take over the host.

Good to know: If the host answers EVERY port (firewall/honeypot), Hades reports one 'unreliable' note instead of dozens of fake open ports.

waf_detect

Firewall / CDN Detection

What it checks: Inspects headers and behaviour to spot a protective layer (Cloudflare, Akamai, AWS, Fastly…) in front of the site.

Consequence of a finding: Confirms a defence layer exists; for an attacker it signals that payloads may be filtered.

Possible attack: Drives WAF-bypass techniques (encoding, fragmentation) so injections still get through.

Good to know: A WAF is good news — but it also explains why other modules may get blocked (403s) during a scan.

tech_stack

Technology Fingerprint

What it checks: Reads headers, HTML, scripts and cookies to identify the web server, language, frameworks, JS libraries and CMS — with version numbers when visible.

Consequence of a finding: Version numbers are gold: an old component with a published vulnerability is often exploitable with a ready-made tool.

Possible attack: Targeted exploitation of a known CVE in an identified, out-of-date component (this module feeds the CVE lookup).

Good to know: Hiding version tokens (Server, X-Powered-By) makes an attacker's job noticeably harder.

js_recon

JavaScript Secret & Endpoint Miner

What it checks: Downloads the site's JavaScript and mines it for leaked secrets (AWS/Google/Stripe/GitHub/Slack keys, private keys) and hidden API endpoints not linked anywhere on the page.

Consequence of a finding: A leaked key is an instant credential; hidden endpoints expose un-advertised, often less-protected functionality.

Possible attack: Direct use of a stolen API key, and attacks against internal/admin API endpoints discovered in the code.

Good to know: Front-end JavaScript is fully public — never put a real secret or a private endpoint in it.

cloud_buckets

Open Cloud Bucket Finder

What it checks: Derives likely S3 / Google Cloud Storage / Azure Blob bucket names from the target and checks whether they exist and are world-readable or listable.

Consequence of a finding: An open bucket can expose backups, user uploads, documents or source — sometimes the entire data store of a company.

Possible attack: Data from cloud storage (MITRE T1530): anyone downloads or lists the bucket's contents directly.

Good to know: Buckets must be private by default; a single mis-set permission has caused many of the largest breaches.

git_dumper

Exposed .git Repository Extractor

What it checks: Detects a publicly reachable /.git/ folder and extracts its metadata — remote URLs, committer emails and the tracked-file list.

Consequence of a finding: An exposed .git lets an attacker reconstruct your entire source code and read its history — including secrets that were committed and 'deleted'.

Possible attack: Full source-code disclosure, then targeted exploitation and harvesting of any hard-coded credentials from history.

Good to know: Block access to .git in the web server, or deploy from outside the web root.

wayback

Wayback / Archive URL Miner

What it checks: Pulls historical URLs and parameters for the domain from the Internet Archive (no traffic to the live site).

Consequence of a finding: Old, forgotten or removed pages and parameters reappear — often still working and less protected than current ones.

Possible attack: Attack-surface expansion: probing archived endpoints/parameters for injection and access-control flaws.

Good to know: Passive recon — it queries the archive, not your server, so it is invisible to your logs.

2 · Website & Misconfiguration Modules (20)

headers_check

HTTP Security Headers Audit

What it checks: Checks the response headers browsers use to enforce security — CSP (directive by directive), HSTS, X-Frame-Options, the cross-origin headers — and flags headers that leak versions.

Consequence of a finding: Missing headers remove browser-side defences, making other attacks far easier to land.

Possible attack: No CSP → easier XSS; no HSTS → downgrade attacks; no X-Frame-Options → clickjacking.

Good to know: These are hardening gaps (mostly Medium): rarely exploitable alone, but they amplify everything else.

robots_txt

robots.txt Analyzer

What it checks: Reads /robots.txt, classifies each disallowed path (admin, backups, .git…), then actively visits the sensitive ones to see if they are really reachable.

Consequence of a finding: robots.txt lists hidden paths in clear text — handing attackers a map of your sensitive areas.

Possible attack: Direct access to a disallowed-but-reachable path (admin, backup, config).

Good to know: A path that returns 404 is a stale leftover; one that returns 200 is a real, reachable target.

sitemap

Sitemap Parser

What it checks: Finds and reads XML sitemaps (including ones declared in robots.txt), follows sitemap indexes one level deep and flags any sensitive URL advertised.

Consequence of a finding: A sitemap that lists internal or admin URLs gives attackers a curated target list with zero guessing.

Possible attack: Direct navigation to interesting endpoints surfaced by the sitemap.

Good to know: Sitemaps are meant to be public; the concern is only when they expose URLs that should stay private.

cms_detect

CMS Detection

What it checks: Fingerprints the content-management system (WordPress, Joomla, Drupal, Magento…) and its version when possible.

Consequence of a finding: Each CMS has a huge catalogue of known plugin/theme vulnerabilities tied to specific versions.

Possible attack: Picking a matching public exploit for the detected CMS/plugin version.

Good to know: WordPress powers a large share of the web and is the most-attacked CMS, so detecting it matters.

admin_panel

Admin / Login Panel Finder

What it checks: Probes known admin paths and verifies by page content whether each is really a login or admin interface (not just a status code).

Consequence of a finding: An exposed admin panel is a prime target; one reachable without logging in can be instant full compromise.

Possible attack: Brute-force / credential-stuffing against the panel, or direct use of an unauthenticated admin UI.

Good to know: Content verification means only genuine login/admin pages are reported, not every page on a catch-all site.

dir_scan

Directory & Path Brute-Forcer

What it checks: Requests a wordlist of common paths to discover what exists, distinguishing open listings, normal paths, auth-protected (401), forbidden (403) and redirects.

Consequence of a finding: Discovered admin areas, backups or config folders are new entry points; an open listing exposes every file.

Possible attack: Forced browsing to hidden functionality and direct download of exposed files.

Good to know: Smart baselines collapse the noise: if the server answers the same to everything, Hades reports one note instead of hundreds of fake hits.

subdomain_scan

Subdomain Enumeration & Takeover

What it checks: Discovers sub-domains actively (DNS brute-force) and passively (Certificate Transparency / crt.sh), then checks each for a dangling-DNS takeover.

Consequence of a finding: Forgotten sub-domains widen the attack surface; a takeover lets an attacker host their own content on YOUR sub-domain.

Possible attack: Subdomain takeover for convincing phishing, plus attacks on weaker dev/staging hosts.

Good to know: A wildcard-DNS check prevents false positives where every name resolves to the same address.

broken_links

Broken Link Checker

What it checks: Checks the status of every crawled link and classifies it — dead (404/410), server error (5xx), access-controlled (401/403) or rate-limited (429).

Consequence of a finding: Dead links hurt usability/SEO; a wall of 403s usually means a WAF is blocking the scan, not broken links.

Possible attack: Minimal direct attack value; mainly hygiene and WAF-behaviour insight.

Good to know: A 403 means 'forbidden', NOT 'broken' — the page likely works fine in a real browser.

http_methods

HTTP Methods Auditor

What it checks: Discovers which HTTP verbs the server accepts and actively verifies the dangerous ones — a safe PUT upload read back, a TRACE echo check (DELETE is advertised-only, never tested).

Consequence of a finding: A confirmed PUT upload is critical; a real TRACE echo can be used to steal headers.

Possible attack: Web-shell upload via PUT (then remote code execution), or Cross-Site Tracing (XST) via TRACE.

Good to know: Only a method proven to work is rated High/Critical; active write tests are skipped in safe mode.

backup_files

Backup & Source File Hunter

What it checks: Looks for backups derived from the target — archive names from the hostname, common backup names, and editor backups of real pages (index.php~, config.php.bak, .swp), confirmed by magic bytes.

Consequence of a finding: A downloadable backup often contains the full source code, a database dump and hard-coded passwords — in one file.

Possible attack: Direct download of source/secrets, leading to full compromise.

Good to know: It confirms a real archive by content, so an HTML 'not found' page is never mistaken for a backup.

sensitive_files

Sensitive File Exposure

What it checks: Probes well-known paths that expose secrets — .env, .git, cloud credentials, SSH keys, framework configs, database dumps — with content checks to avoid false positives.

Consequence of a finding: A readable .env or .git can leak database passwords, API keys and the entire source code.

Possible attack: Credential theft from config files, then direct database/server access.

Good to know: If the server blocks every sensitive path (a good deny-all rule), Hades reports one positive note instead of many false alarms.

cookie_analysis

Cookie Security Flags

What it checks: Inspects the cookies the site sets for the Secure, HttpOnly and SameSite flags (and prefixes like __Host-).

Consequence of a finding: A session cookie missing these flags is far easier to steal or abuse.

Possible attack: Session theft via XSS (no HttpOnly), leakage over HTTP (no Secure), or CSRF (no SameSite).

Good to know: These flags are the difference between an XSS bug being annoying and being account-takeover.

redirect_chain

Redirect Chain Auditor

What it checks: Follows the redirect chain from the target URL and flags HTTPS→HTTP downgrades, off-domain redirects, a missing HTTP→HTTPS upgrade and over-long chains.

Consequence of a finding: A downgrade exposes traffic; an off-domain redirect makes a malicious link look like it starts on your trusted site.

Possible attack: Traffic interception (downgrade) and phishing via open redirect.

Good to know: A clean single HTTP-to-HTTPS redirect is exactly what you want to see.

email_exposure

Exposed Email Finder

What it checks: Collects email addresses from mailto links and page text across the crawl, separating on-domain from third-party addresses.

Consequence of a finding: Published addresses fuel spam and targeted social engineering; an on-domain one is a direct spear-phishing target.

Possible attack: Spear-phishing of named staff and harvesting for credential-stuffing.

Good to know: Use contact forms or obfuscation instead of publishing mailboxes in plain text.

favicon_hash

Favicon Fingerprint

What it checks: Computes a hash of the browser-tab icon (the way Shodan does) so identical icons can be searched across the internet.

Consequence of a finding: Identical favicon hashes quietly reveal shared frameworks, hidden panels or related infrastructure.

Possible attack: Asset discovery — mapping an organisation's other servers/panels by their shared icon.

Good to know: A default framework favicon tells the world what software you run; a custom icon avoids that.

cors_check

CORS Misconfiguration

What it checks: Sends a crafted Origin header to test whether the server reflects any origin AND allows credentials (the dangerous combination).

Consequence of a finding: A permissive policy lets a malicious site read a logged-in victim's authenticated responses from your site.

Possible attack: Cross-origin data theft on behalf of an authenticated victim.

Good to know: 'Allow-Origin: *' is usually fine; reflecting the caller's origin WITH credentials is the real danger.

clickjacking

Clickjacking Verdict

What it checks: Combines X-Frame-Options and CSP frame-ancestors into one 'framable?' answer, weighting risk by whether the page has a login or other forms.

Consequence of a finding: A framable sensitive page can be hidden under a decoy to steal clicks.

Possible attack: Clickjacking — tricking users into clicking real buttons (change settings, confirm actions) through an invisible frame.

Good to know: A framable page with no interactive elements is low impact; one with a login form is high.

dir_listing

Open Directory Listing

What it checks: Detects folders whose entire contents are publicly browsable (the 'Index of /' page), checking crawled folders plus common upload/asset/backup paths.

Consequence of a finding: An open listing hands attackers a full inventory of files — often backups or documents never meant to be public.

Possible attack: Direct browsing and download of every file in the exposed folder.

Good to know: It reuses the crawler's findings, so it tests folders that actually exist rather than guessing.

blacklist_check

Reputation / Blocklist Check

What it checks: Checks the IP and domain against public DNS blocklists (Spamhaus, SpamCop, Barracuda) using the standard key-free protocol.

Consequence of a finding: Being listed signals past spam/malware — or a current compromise — and badly hurts email deliverability.

Possible attack: Not an attack — an early warning that the server may already be hacked or abused.

Good to know: Behind a CDN the checked IP is the shared edge, so a clean result there means less.

screenshot

Homepage Screenshot

What it checks: Opens the homepage in a real headless browser (Chromium) and saves a full-page PNG to screenshots/.

Consequence of a finding: Gives a human a quick visual of the target — useful for triage across many sites.

Possible attack: Not an attack — pure situational awareness.

Good to know: If the browser engine is missing it self-heals or fails gracefully, never breaking the scan.

3 · Vulnerability Detection Modules (12)

sqli_detect

SQL Injection Detection

What it checks: Injects test payloads into URL/form parameters using three techniques — error-based, boolean-blind (TRUE vs FALSE behave differently) and time-based blind (a SLEEP that makes the page hang for a scaling delay).

Consequence of a finding: SQL injection reaches the database directly: dumping all users and passwords, bypassing logins, sometimes taking over the server.

Possible attack: Database exfiltration, authentication bypass, and (with file/OS privileges) remote code execution.

Good to know: It detects, it does not dump; on a hit it shows a ready sqlmap command and can launch it with --exploit. Skipped in safe mode.

xss_detect

Cross-Site Scripting (XSS)

What it checks: Injects a marker, finds exactly where it is reflected (HTML, attribute, JS, comment), then sends the minimal context-specific breakout and checks the special characters survive unencoded.

Consequence of a finding: Reflected XSS runs the attacker's JavaScript in a victim's browser.

Possible attack: Session-cookie theft, actions performed as the victim, page defacement — account takeover when cookie flags are weak.

Good to know: Context analysis keeps it credible: an encoded reflection is Low, a real breakout is High.

command_injection

OS Command Injection

What it checks: Injects shell separators and commands (;id, | whoami, & dir, sleep) and confirms either by the command's OUTPUT appearing, or by a sleep that makes the response hang for a scaling delay.

Consequence of a finding: The attacker runs arbitrary operating-system commands on the server.

Possible attack: Total server compromise — read/modify any file, install a backdoor, pivot into the internal network.

Good to know: Verified not guessed (a delay that scales rules out a slow page); a commix command is attached. Skipped in safe mode.

ssti_detect

Server-Side Template Injection (SSTI)

What it checks: Injects a distinctive maths expression in every template syntax ({{7*7}}, ${7*7}, <%=7*7%>, #{7*7}) and flags it only if the server returns the COMPUTED result instead of the literal text.

Consequence of a finding: The input is evaluated by the template engine — code, not text.

Possible attack: Usually remote code execution: reading secrets/files or taking over the server through the engine.

Good to know: Large numbers make a coincidence impossible; the firing syntax fingerprints the engine. A tplmap command is attached.

lfi_detect

Local File Inclusion / Path Traversal

What it checks: Requests well-known system files via traversal payloads (../../../etc/passwd, ..\windows\win.ini, PHP filter wrappers) and confirms by the file's actual content.

Consequence of a finding: The attacker reads arbitrary server files — configuration, source code, credentials.

Possible attack: Sensitive-file disclosure, chaining to remote code execution via log/session poisoning.

Good to know: Confirmed by file content, never by guesswork, so false positives are rare.

open_redirect

Open Redirect

What it checks: Injects a unique external URL into redirect-style parameters (url, next, return…) and flags when the server forwards the user to that external host (Location, meta refresh or JS).

Consequence of a finding: A link that starts on your trusted domain lands on the attacker's site.

Possible attack: Phishing that abuses your domain's trust, and bypassing redirect allowlists in login/OAuth flows.

Good to know: Severity rises to High when the parameter drives an authentication flow.

ssrf_detect

Server-Side Request Forgery (SSRF)

What it checks: Injects internal and cloud-metadata URLs (127.0.0.1, 169.254.169.254, file://) into URL-shaped parameters and flags responses containing content only the server could fetch.

Consequence of a finding: The server makes requests on the attacker's behalf.

Possible attack: Cloud-credential theft from the metadata service, internal network scanning, reaching never-exposed admin panels.

Good to know: In-band only — truly blind SSRF is caught by the oob_scan profile (out-of-band callbacks).

jwt_attacks

JSON Web Token (JWT) Attacks

What it checks: Finds JWTs (in cookies/headers) and tests them: the 'alg:none' unsigned-token trick, cracking a weak HMAC secret against a wordlist, and reading sensitive data from the claims.

Consequence of a finding: A forgeable or crackable token lets the attacker mint their own valid sessions.

Possible attack: Authentication bypass and privilege escalation by forging a token (e.g. becoming admin).

Good to know: Sign tokens with a strong secret/asymmetric key and reject the 'none' algorithm server-side.

auth_bypass

Access-Control Bypass (401/403)

What it checks: Re-requests forbidden paths with bypass tricks — path mutations (//, /./, %2e), spoofed headers (X-Original-URL, X-Forwarded-For) and HTTP verb tampering.

Consequence of a finding: A protected page becomes reachable without proper authorisation.

Possible attack: Broken access control — viewing or using admin/internal functionality you should not reach.

Good to know: Enforce authorisation in the application, not at the proxy/header layer.

bruteforce

Credential Spraying (opt-in)

What it checks: With the --bruteforce flag only, sprays a small list of common credentials against discovered login forms and HTTP Basic-Auth.

Consequence of a finding: A weak or default password opens a real account.

Possible attack: Account takeover via brute-force / password spraying.

Good to know: Off by default and intrusive — only run it against systems you own or are explicitly authorised to test.

cve_mapping

Known Vulnerability (CVE) Lookup

What it checks: Takes the versions found by tech_stack and asks the official NVD database for matching CVEs, with their CVSS severity score.

Consequence of a finding: A matching CVE often means a ready-made exploit already exists for your component.

Possible attack: Direct exploitation of a public vulnerability in an outdated component.

Good to know: Set an NVD_API_KEY environment variable for a higher rate limit and more complete results.

default_creds

Default Credentials Advisory

What it checks: Fingerprints interfaces famous for default passwords (phpMyAdmin, Tomcat Manager, Jenkins, Grafana…) and lists their documented defaults — without ever logging in.

Consequence of a finding: If a default password was never changed, anyone who finds the panel walks straight in.

Possible attack: Instant access using documented default logins (admin/admin, tomcat/tomcat…).

Good to know: By design it never submits credentials — verify manually on your own systems.

4 · Database Security Audit — db_scan

db_scan

Red-Team Database Security Audit

What it checks: A dedicated profile that scans database ports, fingerprints each engine (MySQL, PostgreSQL, MSSQL, Mongo, Redis, Elasticsearch, CouchDB…), tests for password-less access, and probes parameters for SQL and NoSQL injection.

Consequence of a finding: An open, unauthenticated database is an instant full data breach; an exposed Redis CONFIG becomes a server takeover.

Possible attack: Direct data exfiltration, NoSQL auth-bypass on logins, and Redis→RCE (web shell / SSH key / cron).

Good to know: With --exploit it proves impact — pulls real Redis values / ES & CouchDB docs, an in-band SQLi banner — and writes evidence to loot/. Safe mode skips destructive checks.

db_scan — leak hunting

Secrets, Connection Strings & Admin GUIs

What it checks: Hunts database credentials that leak through the site itself: secret/config files (.env, database.yml, my.cnf, wp-config.php…), hard-coded connection strings in page/JS, GraphQL introspection, exposed admin GUIs and SQL/SQLite dumps.

Consequence of a finding: A readable .env or a leaked connection string IS the password; an exposed admin GUI or dump is a one-click breach.

Possible attack: Credential harvesting then direct database login; schema disclosure via GraphQL introspection.

Good to know: Every credential shown is masked — treat any leaked secret as compromised and rotate it immediately.

db_scan — score, attack path & loot

Exploitation Plan & Extracted Data

What it checks: Assembles the result into a DB Exposure Score (0-100 + grade), an ordered Attack Path of copy-paste exploitation commands (sqlmap, redis-cli, curl…), each tagged with MITRE ATT&CK, and a Loot summary of data actually pulled.

Consequence of a finding: Turns a list of findings into an actionable engagement an operator can reproduce command by command.

Possible attack: Full red-team exploitation workflow against the data tier, with evidence.

Good to know: Opt-in and authorisation-gated — Hades shows the commands; it only runs sqlmap when you pass --exploit and confirm.

5 · AI / LLM Security — ai_scan

ai_scan

AI / LLM Security Audit

What it checks: A dedicated profile that audits the AI layer most scanners ignore: it fingerprints LLM SDKs/providers, hunts exposed AI API keys (sk-ant-…, sk-…, hf_…), finds unauthenticated local LLM servers (Ollama, LM Studio, vLLM…) and exposed AI dev UIs, and locates the prompt-injection surface.

Consequence of a finding: A leaked AI key means billing theft and data access; an open LLM server means free inference and model theft; a live chat endpoint means the model can be manipulated.

Possible attack: Prompt injection (the model obeys attacker instructions), sensitive-information disclosure, and unbounded-consumption (cost) attacks — mapped to OWASP LLM Top 10 & MITRE ATLAS.

Good to know: With --exploit it sends a benign canary to actively confirm prompt injection. Detection-only by default.

ai_scan — prompt injection

The New Attack Surface

What it checks: Detects endpoints where user input reaches a language model and, in active mode, sends a harmless canary instruction ('reply with this token') to see if the model obeys it.

Consequence of a finding: If the model follows injected instructions, an attacker can override its system prompt, exfiltrate data it can see, or bypass its safety rules.

Possible attack: LLM prompt injection (MITRE ATLAS AML.T0051) and system-prompt leakage (OWASP LLM07).

Good to know: Isolate system instructions from user input, constrain and validate model output, and add guardrails.

6 · Active Engagement — engage

engage

Active Exploitation Engagement (auto-pwn)

What it checks: A dedicated offensive profile that first reuses the injection arsenal to confirm bugs, then — once you authorise it — actively proves impact with BENIGN payloads: runs id/uname (RCE proof), reads /etc/passwd (LFI), pulls cloud metadata (SSRF), writing evidence files to loot/.

Consequence of a finding: It converts 'a vulnerability probably exists' into 'here is the proven foothold' — a confirmed foothold is a breach.

Possible attack: Real exploitation of command injection, file inclusion and SSRF, captured as evidence (no destructive actions, no persistence, no DoS).

Good to know: Exploitation-first: choosing the profile asks for authorisation (or pass --exploit). SQL injection still goes through the sqlmap launcher.

7 · Out-of-Band / Blind Vulns — oob_scan

oob_scan

Out-of-Band / Blind Vulnerabilities (OAST)

What it checks: A dedicated profile that catches the bugs which leave NO trace in the HTTP response, by making the server call back to a self-hosted listener. It injects unique-token callback URLs for blind SSRF, blind OS command injection and blind/stored XSS.

Consequence of a finding: Blind bugs are invisible to ordinary scanners yet just as dangerous — and often missed entirely.

Possible attack: Blind SSRF (internal/cloud access), blind RCE (server takeover) and blind/stored XSS — confirmed out-of-band.

Good to know: Works behind NAT: if cloudflared (free, no account) or ngrok is installed Hades auto-opens a public tunnel; otherwise use --oob-host. A callback's source IP is the target itself.

8 · CVE Vulnerability Intelligence — cve_scan

cve_scan

CVE Vulnerability Intelligence (menu option 8)

What it checks: A dedicated profile that fingerprints the target's stack broadly — Server / X-Powered-By headers, tech_stack (JS/CSS/CMS/framework signatures with versions), cms_detect, and SSH/FTP/mail/DB service banners from a port scan — normalises each product to a CPE, and matches real CVEs from a local vulnerability database.

Consequence of a finding: Knowing exactly which published CVEs affect the detected versions turns 'old software' into a concrete, prioritised exploit list.

Possible attack: Maps detected products + versions to known CVEs (nginx, OpenSSH, jQuery, WordPress, Apache, MySQL…), classified CONFIRMED / LIKELY / POSSIBLE by version match.

Good to know: 100% free, no API key: a local SQLite DB built from CISA KEV + FIRST EPSS, with NVD 2.0 queried on demand. Only CVEs from 2020 onward are reported (older ones are filtered out).

cve_scan — prioritisation

KEV + EPSS Priority Score

What it checks: Ranks every matched CVE by a Hades CVE Priority Score (0-100) fusing CVSS severity, FIRST EPSS exploit probability, and whether the CVE is in the CISA KEV catalog (actively exploited in the wild), plus internet exposure and match confidence.

Consequence of a finding: Hundreds of CVEs are noise; the score surfaces the handful that are genuinely exploitable right now.

Possible attack: Focuses the operator on actively-exploited, high-probability CVEs first, instead of raw CVSS alone.

Good to know: Unknown-version products show only the top 10 CVEs (by KEV / EPSS / CVSS) to control noise.

cve_scan — offline corpus

Full Local NVD Database

What it checks: Run tools/build_vulndb.py once to bulk-load the entire NVD corpus (~270k CVEs) into the local SQLite DB. The scan then matches completely offline with no per-product network calls, and the database refreshes incrementally afterwards.

Consequence of a finding: A complete local CVE bank means exhaustive, fast, repeatable matching even with no internet during the engagement.

Possible attack: Offline CVE matching across the full NVD dataset; an optional free NVD_API_KEY speeds the one-time build ~10x.

Good to know: Detection-only intelligence — it reports exposure, it never exploits.

9 · TLS / SSL Attack Surface — tls_scan

tls_scan

Offensive TLS/SSL Attack Surface (menu option 9)

What it checks: A dedicated profile driven by the SSLyze handshake engine. It negotiates TLS the way an attacker probes it and flags legacy protocols (SSLv2/SSLv3, TLS 1.0/1.1), weak/anonymous/NULL cipher suites, missing forward secrecy, certificate trust/expiry/hostname/weak-signature problems, TLS compression (CRIME), and insecure renegotiation.

Consequence of a finding: Transport-layer crypto is what protects every credential and cookie in transit; a downgrade or weak cipher quietly undoes all of it.

Possible attack: Enables on-path downgrade attacks, traffic decryption/sniffing and adversary-in-the-middle (MITRE T1557 / T1040), mapped to CWE-326 / CWE-295 and OWASP A02:2021.

Good to know: Handshake-only and read-only — no exploitation, brute force, DoS or interception. SSLyze is an optional dependency; the module degrades gracefully if it is missing.

tls_scan — known TLS vulns

Heartbleed · ROBOT · CCS Injection

What it checks: Beyond configuration, it runs SSLyze's safe handshake probes for the high-impact TLS vulnerability classes: Heartbleed (CVE-2014-0160), ROBOT (RSA padding oracle), and the OpenSSL ChangeCipherSpec injection flaw (CVE-2014-0224).

Consequence of a finding: These are the bugs that leak private keys or let an attacker decrypt sessions outright — the difference between 'weak' and 'owned'.

Possible attack: Heartbleed leaks process memory (keys/cookies) → Critical; ROBOT/CCS enable RSA decryption / AitM → High.

Good to know: Each finding includes attacker impact, technical evidence and references, and identifies the strongest supported and weakest accepted configuration.

10 · Scoring & Output

scorer

Security Score & Grade

What it checks: Turns all findings into a single 0-100 score and an A-F grade, subtracting points by severity with diminishing returns per module and a confidence weighting.

Consequence of a finding: Your at-a-glance health indicator — a low grade means several real, actionable issues were found.

Possible attack: Not an attack — a prioritisation aid for defenders.

Good to know: INFO findings are context only and NEVER affect the grade; the score reflects genuine problems (Low and above).

HTML report references

Clickable Framework Badges

What it checks: Every finding in the auto-generated HTML report carries reference badges — the CVE, CVSS, CWE, OWASP category, MITRE ATT&CK technique, the relevant RedTeam tool and the matched playbook. Each badge is a link to its canonical explanation page (NVD, the FIRST CVSS calculator, cwe.mitre.org, owasp.org, attack.mitre.org).

Consequence of a finding: One click takes you from a finding to the authoritative definition of the weakness or technique — no copy-pasting IDs into a search engine.

Possible attack: A learning and triage aid: understand and verify every finding in context.

Good to know: Badges open in a new browser tab; the playbook badge opens the full step-by-step skill.